

SAKHLOGGER

Professional Cybersecurity Project Report

Author: SANYAM KUMAR

Technology / Domain: Python, Windows systems programming, keyboard-event monitoring, local file handling, SMTP, and PyInstaller packaging

Tagline: A Windows-focused study of keyboard-event capture, local buffering, startup configuration, and system-information reporting.

Scope and ethics: This report documents the supplied source files as a technical analysis. The implementation captures keyboard input, establishes Windows startup persistence, and sends collected data through SMTP. Such behavior can expose highly sensitive information and must only be tested in an isolated environment with explicit authorization. It must not be deployed against people, devices, accounts, or networks without permission.

1. Project Overview

SAKHLOGGER is the requested project name for the supplied Windows Python utility. The principal source, `Keylogger.py`, installs a global keyboard listener, translates keyboard events into text markers, writes them to a local text file, and periodically prepares the collected data for email delivery. It also attempts to run without a visible console window and registers itself under the current user's Windows startup Run key.

The accompanying `add_startup.py` script is a separate interactive registry utility. It can add an executable to the current user's startup list or remove a corresponding entry. The supplied `code.txt` is an earlier or alternative implementation that logs printable key events to `output.txt` and attempts SMTP delivery every 100 events.

The material does not define a benign business use case or provide evidence of an authorized deployment. Accordingly, this report treats the project as a controlled security-learning artifact and does not claim production readiness.

2. Objectives

The implementation attempts to:

- Capture keyboard events through the Python `keyboard` package.
- Convert special keys such as Space, Enter, Tab, Backspace, and modifier/navigation keys into readable markers.
- Persist captured data locally in a writable text-file location.
- Include host and platform information with an outbound report.

- Trigger email delivery after a buffer threshold or elapsed time.
- Register the main executable or script for current-user startup.
- Prevent multiple simultaneous instances through a named Windows mutex.
- Package `Keylogger.py` as a windowed executable with PyInstaller.

3. Technologies and Tools Used

Technology or tool	Evidence in supplied files	Purpose
Python	<code>Keylogger.py</code> , <code>add_startup.py</code> , <code>code.txt</code>	Application logic and automation
keyboard package	Imported and used in <code>Keylogger.py</code> and <code>code.txt</code>	Keyboard event hook and event callbacks
<code>smtplib</code>	Imported and used in both implementations	SMTP connection and message transmission
<code>winreg</code>	Used by both startup implementations	Access to the Windows Registry Run key
Windows API through <code>ctypes</code>	Used in <code>Keylogger.py</code>	Console hiding and named mutex creation
<code>threading</code>	Used in <code>Keylogger.py</code>	Locking shared keystroke state
<code>socket</code> and <code>platform</code>	Used in <code>Keylogger.py</code> and <code>code.txt</code>	Hostname and operating-system metadata
File I/O and environment APIs	Used throughout the Python files	Local logging, writable-path selection, and temporary debug logging
PyInstaller	<code>Keylogger.spec</code> and <code>build/Keylogger/</code> artifacts	Building the script as <code>Keylogger.exe</code> with no console window
Windows	<code>winreg</code> imports, Windows paths, and Windows API calls	Target operating system
Gmail SMTP endpoint	<code>smtp.gmail.com</code> , port <code>587</code>	Configured email transport using STARTTLS

The source also imports modules such as `sqlite3` , `uuid` , `re` , `os` , and `sys` ; several are not used materially in the primary execution path shown.

4. System Architecture and Workflow



The workflow is based on the behavior in `keylogger.py`. The older `code.txt` flow differs: it writes to `output.txt`, prints status messages, and attempts transmission every 100 events.

5. Implementation

5.1 Startup and process control

At import time, `keylogger.py` calls `hide_console()`. On Windows it obtains the console window handle through `ctypes.windll.kernel32.GetConsoleWindow()` and attempts to hide it with `ShowWindow(..., 0)`.

`check_single_instance()` creates a named global mutex, `Global\WindowsSecurityUpdate_Mutex_2024`. If the Windows last-error code is `183`, the process exits, preventing a second instance from continuing.

The program then selects a writable output location from several user and temporary directories. It creates missing directories, opens the candidate in append mode as a write test, and uses the first successful path. The selected path is written to a temporary debug log named `keylogger_debug.txt`.

5.2 Startup registration

After a five-second delay, `addStartup()` writes the executable or script path to:

```
HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run
```

The registry value name used by the main source is `WindowsSecurityUpdate`. The separate `add_startup.py` utility uses a value name derived from the executable, in the form `Windows<executable-name>Update`, and provides add and remove operations through a text menu.

5.3 Event translation and local logging

`OnKeyboardEvent(event)` reads `event.name`. It maps common special keys as follows:

- `space` becomes a space character.
- `enter` becomes a newline.
- `tab` becomes a tab.
- `backspace` becomes `[BACKSPACE]`.

- Modifier and arrow keys become uppercase bracketed labels such as `[CTRL]`.
- Single-character names are written as the character itself.
- Other names are written in bracketed form.
- `esc` returns `False`, which signals the listener to stop, although the main loop itself is otherwise designed to run continuously.

The translated value is appended to the in-memory `keystroke_buffer` under a thread lock and immediately appended to the selected local output file using UTF-8 encoding.

5.4 Collection and SMTP reporting

The main implementation checks the buffer after each event. It calls `send_email_with_data()` when the buffer contains at least 50 entries or when more than 300 seconds have passed since the last successful send.

The send function reads existing file content, combines it with the in-memory buffer, clears the buffer, truncates the local output file, builds a plain-text message, and connects to `smtp.gmail.com` on port `587`. It requests STARTTLS, logs in with the configured sender account, sends the message to the configured receiver, and records success or failure in the temporary debug log.

The email body includes hostname, platform, version, machine, processor, and timestamp information, followed by the captured keystrokes.

5.5 Packaging

`Keylogger.spec` defines a PyInstaller analysis for `Keylogger.py`, creates a PYZ archive, and builds an executable named `keylogger`. The executable is configured with `console=False`, so it is intended to run without a console window. The supplied `build/Keylogger/` directory contains PyInstaller analysis, executable, package, archive, local-Python, warning, and cross-reference artifacts.

6. Key Features Demonstrated by the Source

- Windows keyboard-event monitoring using `keyboard.on_press`.
- Readable representation of special keys.
- Immediate local persistence of events.
- Thread locking around shared event-buffer operations.
- Time- and count-based transmission triggers.
- Host and operating-system metadata collection.
- Current-user startup registration.
- Single-instance process control.
- Console-window suppression.
- PyInstaller windowed-executable configuration.
- Separate interactive add/remove startup utility.

These are source-level capabilities. The supplied material does not establish that every feature completed successfully during execution.

7. Working Demonstration

Based on the supplied code, the intended execution sequence is:

1. Start `Keylogger.py` directly or run the packaged executable produced from `Keylogger.spec`.
2. Attempt to hide the console window.
3. Create the named mutex and exit if another instance already owns it.
4. Select and initialize a writable `sysdata.txt` path.
5. Wait five seconds and attempt current-user startup registration.
6. Install the keyboard press callback.
7. Translate each event and append the result to memory and local storage.
8. When the count or time threshold is met, prepare and attempt an SMTP report.
9. Clear the local output file after the message is prepared for sending.
10. Continue until the listener is stopped or the process is interrupted.

The older content in `code.txt` describes a console-oriented variant that prints status lines, writes to `output.txt`, and uses a 100-event email trigger.

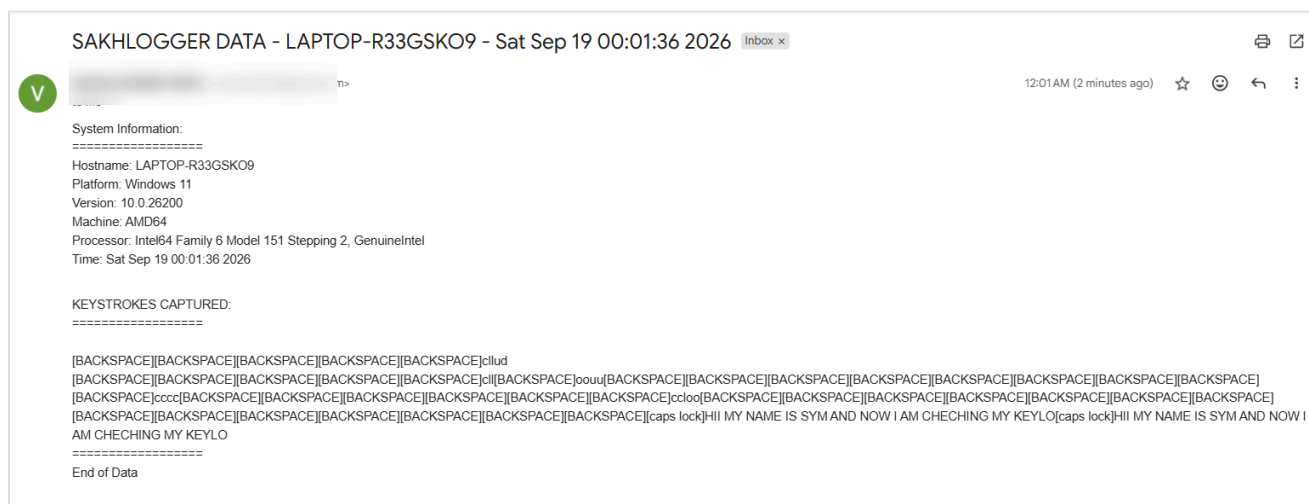
8. Screenshots and Visual Evidence

The project folder contains a supplied image file in `Screenshots/`:

- `SAKHLOGGER 1.png`

The screenshot is retained as supplied project evidence with the visible email address blurred. The report does not assign a specific execution claim to the image because the source package does not provide accompanying annotations or a machine-readable description of its contents. The empty `output.txt` file also does not demonstrate a captured keystroke session.

Figure 1: Supplied project screenshot, preserved without adding unsupported interpretation.



The workflow diagram in Section 4 is conceptual and is clearly labeled as such; it is not a screenshot of a running system.

9. Results

The supplied material demonstrates the following verifiable outcomes:

- A primary Python implementation is present.
- A separate startup-management utility is present.
- A PyInstaller specification targets `keylogger.py` and names the executable `keylogger`.
- PyInstaller build artifacts exist under `build/Keylogger/`.
- A PyInstaller warning file lists mostly optional or platform-specific modules, including POSIX, macOS, and other conditional imports.
- `output.txt` exists but is empty, so no captured data is evidenced there.

No numeric capture count, successful email receipt, verified startup entry, or end-to-end runtime log is supplied. Those results are therefore not claimed.

10. Security and Ethical Considerations

This implementation has high privacy and misuse risk because it captures keyboard input, can persist across user logins, collects system information, and transmits data externally. A captured stream may include passwords, authentication codes, private messages, financial information, and other sensitive data.

The email password is embedded in source code. It should be treated as exposed, revoked or rotated immediately, and replaced with a secure secret-management approach in any authorized defensive laboratory. SMTP configuration should use least privilege, secure credential handling, audit logs, and explicit consent.

Testing must be limited to systems, applications, networks, and data for which proper authorization has been obtained. For portfolio or classroom demonstration, a visible simulator that captures only synthetic test input inside its own window is the safer design.

11. Challenges and Solutions Visible in the Material

Challenge	Implemented response
Output location may not be writable	The main source tests several user and temporary locations before falling back to the current directory.
Concurrent callback and send operations	A <code>threading.Lock</code> protects the shared keystroke buffer.
Duplicate process execution	A named Windows mutex is created and checked.
Special keys are not ordinary characters	The callback maps common control and navigation keys to readable labels.
Packaging as a GUI-style executable	The PyInstaller spec sets <code>console=False</code> .
Startup management needs both add and remove paths	<code>add_startup.py</code> exposes separate add and remove functions through a menu.
Platform-specific packaging warnings	The build warning file records conditional or optional imports reported by PyInstaller.

The material does not include test cases, benchmarks, or a documented debugging session, so further claims about reliability would be speculative.

12. Learning Outcomes

The project demonstrates exposure to:

- Python event-driven callbacks.
- Windows Registry interaction through `winreg`.
- Windows API calls through `ctypes`.
- Process synchronization with a named mutex.
- Thread-safe shared-state handling.
- File encoding and error-tolerant I/O.
- SMTP and STARTTLS concepts.
- Host and platform information APIs.
- PyInstaller specification files and build artifacts.
- The security implications of persistence, monitoring, credential handling, and data exfiltration.

13. Future Improvements

For an ethical and publishable version, the project should be redesigned to:

- Remove global keyboard capture and observe only synthetic input inside a clearly labeled application window.
- Remove startup persistence and hidden-console behavior.
- Remove outbound email and all embedded credentials.

- Add explicit consent, visible status, and a clear stop control.
- Store only non-sensitive test events, with retention limits and user-controlled deletion.
- Add automated unit tests for event translation, path selection, threshold logic, and error handling.
- Add structured logging that excludes captured content and secrets.
- Pin and document dependencies and test the packaged executable in an isolated Windows VM.
- Add code signing and integrity verification for any legitimate Windows utility.

14. Conclusion

SAKHLOGGER provides a detailed example of Windows-specific Python automation involving keyboard callbacks, local file handling, registry startup configuration, system metadata, SMTP, and PyInstaller packaging. The source is technically useful for studying how these mechanisms interact, but its monitoring, persistence, and transmission behaviors create substantial privacy and security concerns. The supplied evidence confirms the project files and build configuration, while it does not confirm a successful end-to-end capture or email delivery. A responsible next iteration should be a visible, consent-based simulator designed for defensive education.